

Phạm Vi Nội Dung Seminar — API & Contract Testing

Nhóm 3 — Lớp cô Hạnh

Môn học: Kiểm thử phần mềm — HCMUS

Ngày tạo: 2026-07-04

Phiên bản: 2.0 (Đã cập nhật & mở rộng)

Dựa trên: Nội dung trao đổi với giảng viên (Script.md) + Nghiên cứu chuyên sâu

Mục lục

- [Tổng quan & Mục tiêu Seminar](#)
- [Phạm Vi Nội Dung — Bản đồ kiến thức](#)
- [Phần A — API Testing \(Lý thuyết + Kỹ thuật\)](#)
- [Phần B — Contract Testing](#)
- [Phần C — Công cụ & Demo \(Postman\)](#)
- [Phần D — Automation & CI/CD](#)
- [Phần E — AI Agent trong API & Contract Testing](#)
- [Phần F — Thực hành \(Hands-on Lab\)](#)
- [Phân công nội dung cho thành viên](#)
- [Danh sách Video Demo cần chuẩn bị](#)
- [Tiêu chí chấm điểm & Checklist hoàn thành](#)

1. Tổng quan & Mục tiêu Seminar

1.1 Định hướng từ giảng viên (trích Script.md)

"Mục tiêu là tìm hiểu ở trong chủ đề testing đó thì người ta sẽ có những loại test gì, những bước gì cần làm... không nên bị giới hạn vào tool."

Seminar cần đạt được:

Tiêu chí	Yêu cầu
Nội dung	Trình bày đầy đủ các kỹ thuật test trong chủ đề — không giới hạn bởi tool

Tiêu chí	Yêu cầu
Tool	Demo nhiều chức năng Postman (Environment, Collection, Variable, Script, Data-driven)
Automation	Export collection → chạy script → tích hợp CI/CD (GitHub Actions)
Độ sâu	Video demo nhiều kịch bản, không chỉ "Hello World 1 phút"
Báo cáo	Viết chi tiết hơn slide — giải thích lý do, ví dụ thực tế

1.2 Sản phẩm cần có

- ☐ **Slide** trình bày tổng quan
- ☐ **Nhiều video demo** (mỗi kịch bản một video)
- ☐ **Báo cáo** viết chi tiết, đầy đủ hơn slide
- ☐ **Repository GitHub** có cấu trúc rõ ràng để học viên clone về thực hành
- ☐ **AI Audit Report** ghi lại toàn bộ tương tác AI

2. Phạm Vi Nội Dung — Bản đồ kiến thức

API & Contract Testing (Seminar W05 - Group 3)

|

|— A. API Testing

| |— A1. Lý thuyết nền (đã có tại API_Testing_Theory.md)

| |— A2. Kỹ thuật thiết kế Test Case

| | |— Functional Testing (Positive/Negative)

| | |— Boundary Value Analysis (BVA)

| | |— Equivalence Partitioning

| | |— Data-Driven Testing (DDT)

- | | └─ Decision Table Testing
- | | └─ Pairwise Testing
- | | └─ State Transition Testing (State Transition Table)
- | | └─ Use Case Testing
- | | └─ Security Testing (OWASP API Top 10)
- | └─ A3. Test theo loại endpoint
 - | └─ Public (No Auth) Endpoints
 - | └─ Authenticated Endpoints
 - | └─ Admin/Role-based Endpoints
- |
- └─ B. Contract Testing
 - | └─ B1. Khái niệm & Khi nào nên dùng
 - | └─ B2. Consumer-Driven Contract Testing (CDCT)
 - | └─ B3. OpenAPI/Swagger Spec & Schema Validation
 - | └─ B4. Bi-directional Contract Testing
 - | └─ B5. Tool: Pact + Pact Broker
- |
- └─ C. Công cụ & Demo – Postman
 - | └─ C1. Collections, Folders, Requests
 - | └─ C2. Environments & Variables
 - | └─ C3. Test Scripts (pm.test, pm.expect)
 - | └─ C4. Pre-request Scripts
 - | └─ C5. Data-Driven Testing (CSV/JSON)
 - | └─ C6. Mock Server & API Documentation
- |
- └─ D. Automation & CI/CD
 - | └─ D1. Newman CLI
 - | └─ D2. Export Collection & GitHub Actions
 - | └─ D3. Test Report (HTML/JUnit)

- |
- | — E. AI Agent trong API & Contract Testing (MỞ RỘNG)
- | | — E1. Landscape: Các loại AI Agent trong testing
- | | — E2. AI Generate Test Cases (Functional + Security)
- | | — E3. AI trong Contract Testing (tự động sinh & verify)
- | | — E4. AI-Powered API Exploration & Fuzzing
- | | — E5. Tích hợp AI vào CI/CD Pipeline
- | | — E6. Prompt Engineering cho API Testing
- | | — E7. Giới hạn & Rủi ro khi dùng AI
- |
- | — F. Thực hành (Hands-on Lab)
- | | — F1. Sample API (mã nguồn mở hoặc tự build)
- | | — F2. Bài tập Postman
- | | — F3. Bài tập Contract Testing (Pact)

3. Phần A — API Testing (Lý thuyết + Kỹ thuật)

A1. Lý thuyết nền (đã có tại [API_Testing_Theory.md](#))

Xem chi tiết tại [API_Testing_Theory.md](#). Bao gồm:

- Khái niệm API, HTTP Request/Response
- HTTP Methods, Status Codes
- Các loại Authentication (No Auth → Basic → API Key → JWT → OAuth 2.0 → mTLS)
- Phân loại API (REST, SOAP, GraphQL, gRPC)
- OWASP API Security Top 10 (2023)

A2. Kỹ thuật thiết kế Test Case API (QUAN TRỌNG)

A2.1 Functional Testing

Positive Testing (Happy Path)

Kiểm tra API hoạt động đúng với **đầu vào hợp lệ**:

Loại kiểm tra	Mô tả	Ví dụ
Status Code	Đúng code với từng scenario	POST → 201, GET → 200, DELETE → 204
Response Body	Đúng schema, đúng dữ liệu	id là integer, email có định dạng đúng
Response Headers	Có đủ headers cần thiết	Content-Type: application/json
Data Persistence	Dữ liệu thực sự được lưu	Tạo user → GET user → kiểm tra trùng khớp

Ví dụ test case Postman (pm.test):

```
// Test trong tab "Tests" của Postman
pm.test('Status code is 201', () => {
  pm.response.to.have.status(201);
});

pm.test('Response has required fields', () => {
  const json = pm.response.json();
  pm.expect(json).to.have.property('id');
  pm.expect(json).to.have.property('email');
  pm.expect(json.email).to.include('@');
});

pm.test('Response time < 2000ms', () => {
  pm.expect(pm.response.responseTime).to.be.below(2000);
});
```

Negative Testing (Error Path)

Kiểm tra API xử lý đúng với **đầu vào không hợp lệ**:

Scenario	Input	Expected
Thiếu field bắt buộc	<code>{"name": "A"}</code> (thiếu email)	400 Bad Request
Sai kiểu dữ liệu	<code>{"age": "abc"}</code> (string thay int)	400/422
Vượt giới hạn ký tự	<code>{"name": "A" × 256}</code>	400/422
Null/Empty values	<code>{"email": ""}</code>	400/422
Method không hỗ trợ	DELETE /api/products (nếu không hỗ trợ)	405 Method Not Allowed
Endpoint không tồn tại	GET /api/nonexistent	404 Not Found
Duplicate resource	POST tạo email đã tồn tại	409 Conflict

A2.2 Boundary Value Analysis (BVA) áp dụng cho API

BVA không chỉ dành cho UI — áp dụng trực tiếp vào API test với các trường có giới hạn:

Field	Min	Max	Test points
age	0	120	-1, 0, 1, 119, 120, 121

Field	Min	Max	Test points
name (max 50 chars)	1	50	0 chars, 1 char, 49 chars, 50 chars, 51 chars
price	0.01	999999	0, 0.01, 999999, 1000000
page (pagination)	1	—	0, 1, max_int
limit	1	100	0, 1, 100, 101

Ví dụ test case:

```
# BVA cho pagination
GET /api/products?page=0&limit=10 → 400 Bad Request (page min = 1)
GET /api/products?page=1&limit=10 → 200 OK (valid)
GET /api/products?page=1&limit=0 → 400 Bad Request (limit min = 1)
GET /api/products?page=1&limit=101 → 400 Bad Request (limit max = 100)
```

A2.3 Equivalence Partitioning cho API

Phân chia các nhóm đầu vào tương đương:

```
Input: email field
├─ Hợp lệ: "user@example.com", "test.user+tag@domain.co"
├─ Không có @: "userexample.com"
├─ Thiếu domain: "user@"
├─ Thiếu local: "@example.com"
├─ Empty: ""
```

└─ Null: null (hoặc field bị bỏ qua)

A2.4 Data-Driven Testing (DDT)

Kỹ thuật dùng **dữ liệu ngoài** để chạy cùng một test script với nhiều bộ dữ liệu:

File CSV cho Postman:

```
username,password,expected_status
admin,Admin@123,200
user1,Pass@123,200
,Pass@123,400
admin,,400
wrong_user,wrong_pass,401
admin,expired_pass,401
```

Chạy Data-Driven trong Postman:

```
# Newman với data file
newman run collection.json -e env.json -d testdata.csv --reporters cli,htmlextra
```

Khi nào dùng DDT:

- Kiểm tra nhiều user/role khác nhau
- Test form validation với nhiều bộ input
- Test CRUD với nhiều record
- Test permission matrix (user × endpoint × action)

A2.4.1 Decision Table Testing (Bảng quyết định) áp dụng cho API

Kỹ thuật kiểm thử tổ hợp các điều kiện đầu vào để kiểm tra logic nghiệp vụ phức tạp của API.

- **Ví dụ áp dụng:** API thanh toán đơn hàng POST /api/v1/checkout.
- **Các tham số đầu vào:** `Cart_Valid` (Giỏ hàng hợp lệ), `Promo_Valid` (Mã giảm giá hợp lệ - True/False/None), `In_Stock` (Đủ tồn kho).
- **Kết quả kiểm thử:** HTTP Status Code, trạng thái tạo đơn hàng, áp dụng mã giảm giá.

Bảng quyết định mẫu:

Conditions / Actions	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6
Giỏ hàng hợp lệ (Cart Valid)?	False	True	True	True	True	True
Mã giảm giá hợp lệ (Promo Valid)?	N/A	False	True	True	None	None
Còn đủ tồn kho (In Stock)?	N/A	N/A	False	True	True	False
HTTP Status Code	400	422	409	200	200	409
Tạo đơn hàng thành công?	No	No	No	Yes	Yes	No
Áp dụng giảm giá?	No	No	No	Yes	No	No

A2.4.2 Pairwise Testing (Kiểm thử cặp) áp dụng cho API

Giảm thiểu số lượng kịch bản kiểm thử khi API có quá nhiều Query Parameters/Request Body parameters kết hợp.

- **Ví dụ áp dụng:** API tìm kiếm sản phẩm GET /api/products/search với 4 tham số: category (3 giá trị: Electronics, Clothing, Home), price_range (2 giá trị: Low, High), sort (2 giá trị: Popularity, Price), shipping (2 giá trị: Free, Standard).
- **Số tổ hợp đầy đủ:** $3 \times 2 \times 2 \times 2 = 24$ test cases.
- **Số tổ hợp Pairwise:** Giảm xuống chỉ còn **9 test cases** nhưng vẫn bao phủ tất cả các cặp tương tác.

TC ID	category	price_range	sort	shipping
TC-01	Electronics	Low	Popularity	Free
TC-02	Electronics	High	Price	Standard
TC-03	Clothing	Low	Price	Free
TC-04	Clothing	High	Popularity	Standard
TC-05	Home	Low	Popularity	Standard
TC-06	Home	High	Price	Free
TC-07	Electronics	Low	Price	Standard
TC-08	Clothing	High	Popularity	Free
TC-09	Home	High	Popularity	Free

A2.4.3 State Transition Testing (Kiểm thử chuyển trạng thái) áp dụng cho API

Kiểm thử hoạt động của API dựa trên vòng đời và trạng thái hiện tại của tài nguyên trong database.

- **Ví dụ áp dụng:** API quản lý vòng đời đơn hàng (CREATED, PAID, SHIPPED, DELIVERED, CANCELLED).
- **Kịch bản kiểm thử:**
- *Chuyển đổi hợp lệ:* Đơn hàng PAID → gọi API giao hàng POST /orders/ship → chuyển sang SHIPPED (Trả về 200 OK).
- *Chuyển đổi không hợp lệ:* Đơn hàng chưa thanh toán CREATED → cố tình gọi API giao hàng POST /orders/ship → Server từ chối chuyển trạng thái (Trả về 400 Bad Request hoặc 409 Conflict).

stateDiagram-v2

[*] --> CREATED : POST /orders

CREATED --> PAID : POST /orders/pay

PAID --> SHIPPED : POST /orders/ship

SHIPPED --> DELIVERED : POST /orders/deliver

CREATED --> CANCELLED : POST /orders/cancel

A2.4.4 Use Case Testing (Kiểm thử kịch bản / API Chaining) áp dụng cho API

Kiểm thử liên chuỗi các API kết hợp (API Chaining) để mô phỏng một luồng quy trình nghiệp vụ thực tế của người dùng từ đầu đến cuối.

- **Ví dụ kịch bản mua hàng trọn vẹn:**
- POST /api/auth/register (Tạo tài khoản)
- POST /api/auth/login (Đăng nhập → trích xuất Bearer Token)
- GET /api/products (Tìm kiếm sản phẩm)
- POST /api/cart (Thêm sản phẩm vào giỏ hàng → trích xuất Cart ID)
- POST /api/checkout (Thực hiện đặt thanh toán và kiểm tra trạng thái đơn)
- **Kỹ thuật demo:** Sử dụng Postman Environment Variables để truyền dữ liệu động từ Response của API trước vào Request của API sau.

A2.5 Test Case cho Authenticated Endpoints

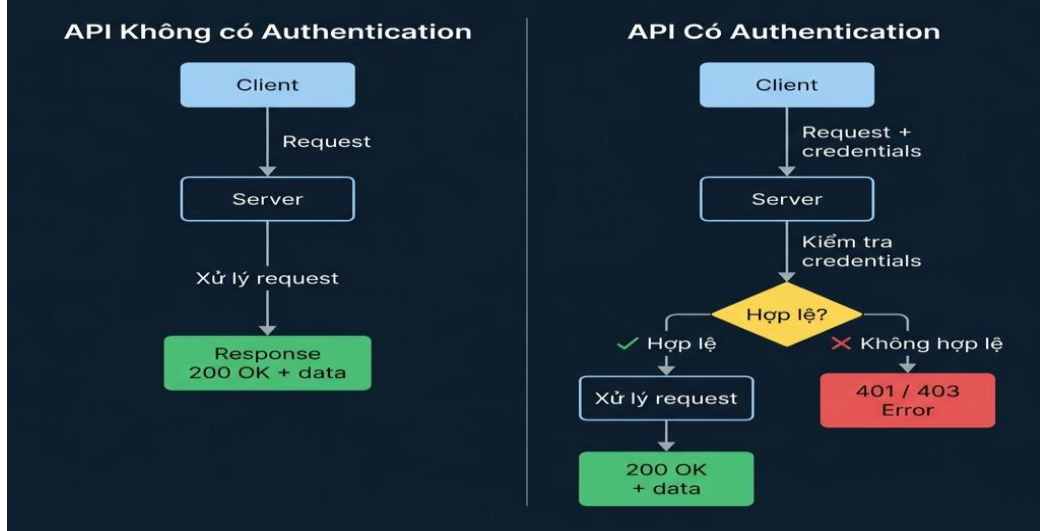
Mà trận kiểm thử Authentication × Authorization:

TC ID	Scenario	Token	Endpoint	Expected
TC-AUTH-01	Không có token	None	/api/orders	401 Unauthorized
TC-AUTH-02	Token hết hạn	<expired>	/api/orders	401 Unauthorized

TC ID	Scenario	Token	Endpoint	Expected
TC-AUTH-03	Token giả mạo	invalid123	/api/orders	401 Unauthorized
TC-AUTH-04	Token hợp lệ — user	<user_token>	/api/orders	200 OK
TC-AUTH-05	User xem tài nguyên user khác	<user_A_token>	/api/orders/user_B_id	403 Forbidden
TC-AUTH-06	User gọi admin endpoint	<user_token>	/admin/users	403 Forbidden
TC-AUTH-07	Admin gọi admin endpoint	<admin_token>	/admin/users	200 OK
TC-AUTH-08	Token đúng nhưng role không đủ	<moderator_token>	/admin/delete	403 Forbidden

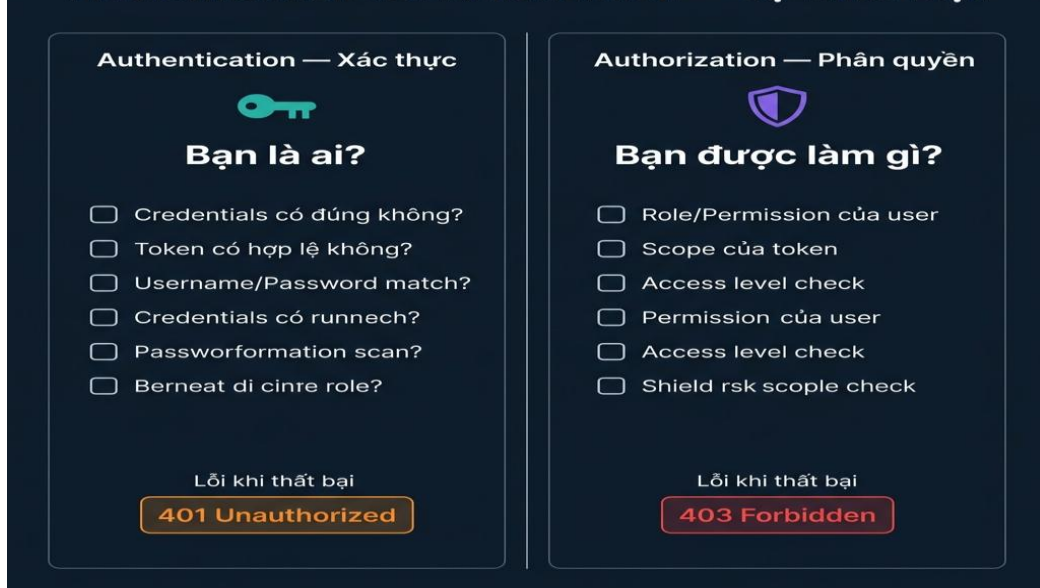
Sơ đồ minh họa kiểm thử Xác thực & Phân quyền API:

So Sánh Luồng Xử Lý API: Có vs. Không Có Authentication



(Luồng xử lý yêu cầu API giữa trường hợp có xác thực và không xác thực - [Mã nguồn Mermaid](#))

Authentication vs Authorization — Sự Khác Biệt



(Sự khác biệt giữa Authentication - Kiểm tra danh tính và Authorization - Kiểm tra quyền hạn - [Mã nguồn Mermaid](#))

A2.6 Security Testing (OWASP API Top 10 — 2023)

Tham khảo: [OWASP API Security Top 10 \(2023\)](#)

Test cases cụ thể cần demo:

Vulnerability	Test Case	Cách test
BOLA/IDOR	Thay đổi id trong URL của user khác	GET /api/orders/999 với token của user khác
Broken Auth	Token bypass, weak JWT	Gửi token đã sửa payload
Mass Assignment	Gửi thêm field không được phép	{"role": "admin"} trong request update profile
Rate Limiting	Gửi 100 request liên tiếp	Loop request → kiểm tra 429 Too Many Requests
SQL Injection	Inject trong query param	?id=1 OR 1=1
SSRF	Gửi URL nội bộ trong body	{"callback_url": "http://internal-server/secret"}

Script demo JWT Token Manipulation (Postman Pre-request):

```
// Giả lập attacker thay đổi payload trong JWT
// (chỉ dùng để demo – JWT thật sẽ invalid vì signature sai)
const fakePayload = {
  sub: '999', // thay đổi user ID sang ID khác
  role: 'admin', // nâng quyền
  exp: 9999999999,
};

const fakePayloadB64 = btoa(JSON.stringify(fakePayload));
```

```
const [header, , signature] = pm.environment.get('authToken').split('.');

const tamperedToken = `${header}.${fakePayloadB64}.${signature}`;

pm.environment.set('tamperedToken', tamperedToken);

// => Expected: 401 Unauthorized (server từ chối vì signature không match)
```

Script demo Rate Limiting (Postman Pre-request loop):

```
// Gửi 20 request liên tiếp trong Collection Runner → kiểm tra 429
// File: testdata.csv – tạo 20 dòng rỗng với expected_status = 429 (từ dòng 11 trở đi)
// Script test:

const iteration = pm.info.iteration; // số lần lặp hiện tại

if (iteration <= 10) {

    pm.test('Rate not exceeded yet', () => pm.response.to.have.status(200));

} else {

    pm.test('Rate limit triggered', () => pm.response.to.have.status(429));

    pm.test('Retry-After header present', () => {

        pm.expect(pm.response.headers.has('Retry-After')).to.be.true;

    });

}
```

Script demo Mass Assignment Attack:

```
# Attacker gửi thêm field "role" vào request update profile

PATCH /api/v1/users/42

Authorization: Bearer <user_token>

Content-Type: application/json

{

    "name": "Nguyen Van A",

    "email": "a@example.com",

    "role": "admin",
```

```
"is_verified": true,
"credit": 999999
}
// Test script – kiểm tra server không chấp nhận field không được phép
pm.test('Server ignores unauthorized fields', () => {
  const json = pm.response.json();
  pm.expect(json.role).to.equal('user'); // role không bị thay đổi
  pm.expect(json.credit).to.not.equal(999999); // credit không bị inject
});
```

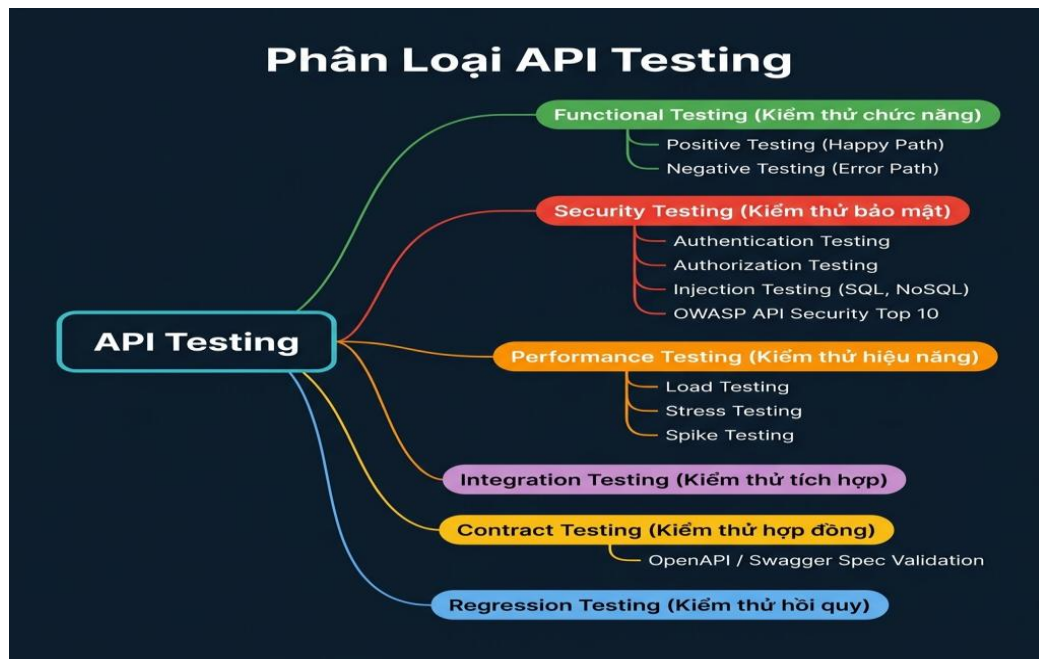
A3. Test theo loại Endpoint

Phân loại endpoint cần test

API Endpoints

- |— Public (No Auth Required)
 - | — Test: Đúng data, rate limiting, method không hỗ trợ
 - |
- |— Authenticated (JWT/Token Required)
 - | — Test: No token, expired, invalid, valid + IDOR
 - |
- |— Role-Based (RBAC)
 - | — Test: User vs Admin vs Moderator quyền hạn khác nhau
 - |
- |— Admin (Super Privileged)
 - | — Test: Chặn hoàn toàn non-admin, audit log

Sơ đồ phân loại tổng quan kỹ thuật kiểm thử API:



(Bản đồ phân loại kỹ thuật kiểm thử API theo chức năng, bảo mật và hiệu năng - [Mã nguồn Mermaid](#))

4. Phần B — Contract Testing

B1. Khái niệm & Vấn đề cần giải quyết

Vấn đề trong kiến trúc Microservices

Khi hệ thống có nhiều service giao tiếp với nhau, một thay đổi nhỏ ở **Provider** (server) có thể làm vỡ **Consumer** (client):

Không có Contract Testing:

Provider thay đổi response →

Consumer nhận data sai →

Bug chỉ phát hiện ở môi trường integration →

Chi phí fix cao

Contract Testing giải quyết vấn đề này

Có Contract Testing:

Consumer định nghĩa kỳ vọng (Contract) →

Provider verify contract →

Nếu vi phạm → CI fail ngay lập tức →

Bug phát hiện sớm, chi phí thấp

B2. Consumer-Driven Contract Testing (CDCT) với Pact

B2.1 Khái niệm cốt lõi

Thuật ngữ	Nghĩa
Consumer	Service/client sử dụng API (frontend, mobile, service A)
Provider	Service cung cấp API (backend, service B)
Contract/Pact	File JSON mô tả kỳ vọng của Consumer với Provider
Pact Broker	Kho lưu trữ trung tâm cho các contracts
Can-I-Deploy	Tool kiểm tra tính tương thích trước khi deploy
Provider State	Điều kiện tiên quyết trên server trước khi verify (ví dụ: user tồn tại)
Interaction	Một cặp Request + Expected Response trong contract
Matcher	Quy tắc so sánh linh hoạt (type, regex, like) thay vì so sánh exact

B2.2 Luồng CDCT

1. Consumer viết test → sinh ra Pact file (contract)
2. Consumer publish Pact lên Pact Broker
3. Provider pull Pact từ Broker
4. Provider verify từng interaction trong Pact
5. Kết quả verification được publish lại Broker
6. Can-I-Deploy kiểm tra: Consumer + Provider có tương thích không?
7. Nếu OK → Deploy

Biểu đồ trình tự luồng kiểm thử hợp đồng CDCT (4 bước):

sequenceDiagram

autonumber

participant Consumer as Consumer (Client/CI)

participant Broker as Pact Broker

participant Provider as Provider (Backend/CI)

rect rgb(230, 245, 255)

Note over Consumer: Bước 1: Phát triển & Chạy Test

Consumer->>Consumer: Khởi tạo Mock Provider cục bộ

Consumer->>Consumer: Gọi API client thật vào Mock

Consumer->>Consumer: Xác minh Client xử lý đúng response mock

Consumer->>Consumer: Tự động xuất ra file contract (pact JSON)

end

rect rgb(235, 250, 235)

Note over Consumer, Broker: Bước 2: Đẩy Hợp Đồng

Consumer->>Broker: Publish file pact JSON kèm số phiên bản (version)

end

```
rect rgb(255, 245, 230)
```

```
Note over Broker, Provider: Bước 3: Xác minh Hợp Đồng
```

```
Provider->>Broker: Tải file pact JSON của Consumer về
```

```
Provider->>Provider: Thiết lập State (trạng thái data ban đầu)
```

```
Provider->>Provider: Pact Verifier phát lại Request trong file Pact vào API thật
```

```
Provider->>Provider: So sánh Response thật với Response kỳ vọng trong file Pact
```

```
end
```

```
rect rgb(250, 235, 250)
```

```
Note over Provider, Broker: Bước 4: Trả Kết Quả
```

```
Provider->>Broker: Gửi kết quả xác minh (Verification Results: Pass/Fail)
```

```
end
```

B2.2.1 Code thực tế: Consumer viết Pact test (JavaScript — Pact JS)

```
// consumer.pact.spec.js – Consumer side

const { PactV3, MatchersV3 } = require('@pact-foundation/pact');
const { like, eachLike, string, integer, regex } = MatchersV3;

const provider = new PactV3({
  consumer: 'web-frontend',
  provider: 'user-service',
  dir: './pacts', // Pact file sẽ được sinh ra ở đây
});

describe('User Service Contract', () => {
  it('should return user profile for valid ID', () => {
    // 1. Định nghĩa interaction kỳ vọng
    provider
```

```

    .given('user with ID 42 exists') // Provider State
    .uponReceiving('a GET request for user 42')
    .withRequest({
      method: 'GET',
      path: '/users/42',
      headers: { Authorization: like('Bearer valid-token') }, // dùng matcher
    })
    .willRespondWith({
      status: 200,
      headers: { 'Content-Type': 'application/json' },
      body: {
        id: integer(42), // matcher: chỉ cần là integer
        name: string('Nguyen Van A'), // matcher: chỉ cần là string
        email: regex('\\S+@\\S+\\.\\S+', 'a@example.com'), // regex match
        roles: eachLike('user'), // matcher: mảng ít nhất 1 phần tử
      },
    });

// 2. Chạy actual HTTP call trong mock server
return provider.executeTest(async (mockServer) => {
  const client = new UserServiceClient(mockServer.url);
  const user = await client.getUser(42);

  // 3. Assert kết quả
  expect(user.id).toBe(42);
  expect(user.email).toMatch(/@/);

  // => Pact tự động sinh ra file ./pacts/web-frontend-user-service.json
});
});

```

```
});
```

B2.2.2 Code thực tế: Provider verify Pact (JavaScript)

```
// provider.pact.spec.js – Provider side
const { PactV3 } = require('@pact-foundation/pact');

describe('User Service Provider Verification', () => {
  it('should verify contracts from Pact Broker', () => {
    return new PactV3({
      provider: 'user-service',
      providerBaseUrl: 'http://localhost:3000', // URL server thật
      pactBrokerUrl: 'https://your-broker.pactflow.io',
      pactBrokerToken: process.env.PACT_BROKER_TOKEN,
      publishVerificationResult: true,
      providerVersion: process.env.GIT_SHA,

      // Setup Provider States – chuẩn bị data trước khi verify
      stateHandlers: {
        'user with ID 42 exists': async () => {
          // Seed database hoặc mock repository
          await db.users.upsert({
            id: 42,
            name: 'Nguyen Van A',
            email: 'a@example.com',
          });
        },
        'no users exist': async () => {
          await db.users.deleteAll();
        },
      },
    });
  });
});
```

```
    },  
    }).verifyProvider();  
  });  
});
```

B2.3 Ví dụ Pact Contract (JSON)

```
{  
  "consumer": { "name": "web-frontend" },  
  "provider": { "name": "user-service" },  
  "interactions": [  
    {  
      "description": "a request for user profile",  
      "request": {  
        "method": "GET",  
        "path": "/users/42",  
        "headers": { "Authorization": "Bearer valid-token" }  
      },  
      "response": {  
        "status": 200,  
        "headers": { "Content-Type": "application/json" },  
        "body": {  
          "id": 42,  
          "name": "Nguyen Van A",  
          "email": "a@example.com"  
        }  
      }  
    }  
  ]  
}
```

B2.4 So sánh: E2E Testing vs Contract Testing

Tiêu chí	E2E Testing	Contract Testing (Pact)
Tốc độ phản hồi	Chậm (cần full environment)	Nhanh (chạy isolated)
Độ ổn định	Dễ flaky (network, data)	Ổn định, deterministic
Phát hiện lỗi	Muộn (integration env)	Sớm (trong CI của từng service)
Debug	Khó xác định nguồn lỗi	Rõ ràng: consumer/provider nào sai
Deploy	Phải deploy cùng lúc	Independent deployment
Chi phí	Cao	Thấp

B3. OpenAPI/Swagger Specification & Schema Validation

B3.1 API Design-First Approach

Thay vì viết code trước rồi mới tạo doc, **Design-First** đặt API spec làm trung tâm:

Design-First Workflow:

Thiết kế OpenAPI Spec → Tạo Mock Server →

Consumer phát triển dựa vào Mock →

Provider implement theo Spec →

Contract Test verify sự tương thích

B3.2 Ví dụ OpenAPI Spec (YAML)


```
openapi: 3.0.3
info:
  title: User API
  version: 1.0.0

paths:
  /users/{id}:
    get:
      summary: Get user by ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        "200":
          description: User found
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/User"
        "404":
          description: User not found

components:
  schemas:
    User:
      type: object
```

```
required: [id, name, email]

properties:

id:

type: integer

name:

type: string

maxLength: 100

email:

type: string

format: email
```

B3.3 Schema Validation trong Postman

```
// Validate response schema bằng Ajv trong Postman

const schema = {
  type: 'object',
  required: ['id', 'name', 'email'],
  properties: {
    id: { type: 'integer' },
    name: { type: 'string' },
    email: { type: 'string', format: 'email' },
  },
};

pm.test('Response matches schema', () => {
  pm.response.to.have.jsonSchema(schema);
});
```

B4. Bi-Directional Contract Testing

Phương pháp mới hơn, ít phức tạp hơn CDCT truyền thống:

Consumer cung cấp: Expected OpenAPI spec (mock server spec)

Provider cung cấp: Actual OpenAPI spec (implementation spec)

PactFlow/Tool so sánh hai spec →

Kiểm tra compatibility →

Không cần viết Pact test từ đầu

Ưu điểm: Tận dụng OpenAPI spec có sẵn, ít boilerplate code hơn.

Dùng khi: Team đã có OpenAPI spec nhưng chưa quen với Pact.

B5. Khi nào NÊN và KHÔNG NÊN dùng Contract Testing

Tình huống	Khuyến nghị
Microservices nhiều team độc lập	NÊN dùng
Deployment độc lập (independent)	NÊN dùng
Frontend + Backend phát triển song song	NÊN dùng
API public (nhiều consumer ngoài)	NÊN dùng
Monolith, 1 team	Không cần thiết
Internal function calls (không qua network)	Không phù hợp

Tình huống	Khuyến nghị
Testing business logic phức tạp	Dùng unit/integration test

5. Phần C — Công cụ & Demo (Postman)

C1. Tổng quan Postman Workspace

Postman

```

├─ Workspace (Organization level)
|
| └─ Collections (Test Suites)
|   |
|   | └─ Folders (Nhóm endpoint theo feature)
|   |
|   | └─ Requests (Individual API calls)
|   |
|   └─ Environments (dev/staging/prod config)
|
| └─ Variables (Global, Collection, Env, Local)
|
| └─ Mock Servers
|
└─ API Documentation
  
```

C2. Environments & Variables

Tại sao cần Environment?

- Không hardcode URL/token vào từng request
- Dễ chuyển đổi giữa dev/staging/production

Phân cấp Variables:

Level	Scope	Priority
Global Variables	Toàn bộ workspace	Thấp nhất
Collection Variables	Trong một Collection	Trung bình
Environment Variables	Khi chọn Environment này	Cao
Local Variables	Chỉ trong 1 request/script	Cao nhất

Ví dụ Environment:

```
// Environment: "Development"
{
  "baseUrl": "http://localhost:3000/api/v1",
  "authToken": "eyJhbGciOi...",
  "adminToken": "eyJhbGciOi..."
}

// Environment: "Staging"
{
  "baseUrl": "https://staging.example.com/api/v1",
  "authToken": "staging-token-here",
  "adminToken": "staging-admin-token"
}
```

Sử dụng trong request:

```
URL: {{baseUrl}}/users
```

Header: Authorization: Bearer {{authToken}}

C3. Test Scripts (pm API)

Các assertion phổ biến:

```
// --- Status Code ---
pm.test('Status 200', () => pm.response.to.have.status(200));

// --- Response Time ---
pm.test('Fast response', () => pm.expect(pm.response.responseTime).to.be.below(500));

// --- JSON Body ---
const json = pm.response.json();
pm.test('Has id field', () => pm.expect(json).to.have.property('id'));
pm.test('Email format', () => pm.expect(json.email).to.match(/.+@.+\.+\.+\/));

// --- Headers ---
pm.test('Content-Type JSON', () => {
  pm.expect(pm.response.headers.get('Content-Type')).to.include('application/json');
});

// --- Lưu giá trị vào variable (dùng ở request sau) ---
const token = json.access_token;
pm.environment.set('authToken', token);

// --- Schema Validation ---
const schema = { type: 'object', required: ['id', 'name'] };
pm.test('Valid schema', () => pm.response.to.have.jsonSchema(schema));
```

C4. Pre-request Scripts

Dùng để chuẩn bị dữ liệu **trước khi** gửi request:

```
// Auto-generate unique test data
const timestamp = new Date().getTime();
pm.environment.set('testEmail', `test_${timestamp}@example.com`);
pm.environment.set('testUsername', `user_${timestamp}`);

// Tự động login và lấy token nếu token hết hạn
const tokenExpiry = pm.environment.get('tokenExpiry');
if (!tokenExpiry || Date.now() > tokenExpiry) {
  pm.sendRequest(
    {
      url: pm.environment.get('baseUrl') + '/auth/login',
      method: 'POST',
      body: {
        mode: 'raw',
        raw: JSON.stringify({ username: 'admin', password: 'Admin@123' }),
      },
    },
    (err, res) => {
      pm.environment.set('authToken', res.json().access_token);
      pm.environment.set('tokenExpiry', Date.now() + 3600000); // 1 hour
    },
  );
}
```

C5. Data-Driven Testing

Bước thực hiện:

1. Tạo file testdata.csv:

```
email,password,expected_status,description
admin@test.com,Admin@123,200,Valid admin login
user@test.com,User@123,200,Valid user login
,,400,Missing credentials
admin@test.com,wrongpass,401,Wrong password
notexist@test.com,Pass@123,401,User not found
```

1. Sử dụng biến trong Postman Request:

```
Body: {"email": "{{email}}", "password": "{{password}}"}"
```

1. Test script đọc expected value:

```
pm.test(`Status should be ${pm.iterationData.get('expected_status')}`, () => {
  pm.response.to.have.status(parseInt(pm.iterationData.get('expected_status')));
});
```

1. Chạy Collection Runner với Data File:
2. Postman → Collection → Run → chọn file CSV → Iterate

C6. Mock Server

Dùng khi API chưa được implement — Consumer có thể test trước:

Postman Mock Server:

1. Định nghĩa response mong muốn cho mỗi endpoint
2. Postman tạo ra URL mock: <https://xxxxx.mock.pstmn.io>
3. Consumer gọi vào URL mock → nhận response đã định nghĩa

4. Provider implement xong → switch baseUrl từ mock sang real

6. Phần D — Automation & CI/CD

D1. Newman CLI

Cài đặt và sử dụng cơ bản:

```
# Cài đặt
npm install -g newman newman-reporter-htmlextra

# Export từ Postman: Collections & Environments dưới dạng JSON

# Chạy collection
newman run collection.json -e environment.json

# Chạy với data file
newman run collection.json -e env.json -d testdata.csv

# Chạy với HTML report
newman run collection.json -e env.json -r cli,htmlextra \
  --reporter-htmlextra-export reports/report.html

# Chạy với JUnit report (cho CI)
newman run collection.json -e env.json -r cli,junit \
  --reporter-junit-export reports/junit.xml

# Stop on first failure
newman run collection.json --bail
```

```
# Chạy parallel (nhiều collection cùng lúc – dùng trong CI)

npm install -g newman-reporter-htmlextra p-limit # hoặc dùng GNU parallel

# PowerShell: chạy 2 collection song song

$job1 = Start-Job { newman run auth-tests.json -e env.json -r cli,junit --reporter-junit-export reports/auth.xml }

$job2 = Start-Job { newman run product-tests.json -e env.json -r cli,junit --reporter-junit-export reports/product.xml }

Wait-Job $job1, $job2
```

D2. Tích hợp GitHub Actions

File `.github/workflows/api-tests.yml`:

```
name: API Tests

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  api-test:
    runs-on: ubuntu-latest

    # — Environment Matrix: chạy trên cả dev và staging —
    strategy:
      matrix:
        environment: [dev, staging]

    fail-fast: false # không dừng staging nếu dev fail
```

steps:

- name: Checkout code

uses: actions/checkout@v4

- name: Setup Node.js

uses: actions/setup-node@v4

with:

node-version: '20'

- name: Install Newman

run: npm install -g newman newman-reporter-htmlextra

- name: Run API Tests (\${{ matrix.environment }})

run: |

newman run postman/collection.json \

-e postman/env-\${{ matrix.environment }}.json \

--reporters cli,htmlextra,junit \

--reporter-htmlextra-export reports/report-\${{ matrix.environment }}.html \

--reporter-junit-export reports/junit-\${{ matrix.environment }}.xml \

--bail

env:

AUTH_TOKEN: \${{ secrets[format('API_AUTH_TOKEN_{0}', matrix.environment)] }}

- name: Upload Test Report

uses: actions/upload-artifact@v4

if: always()

with:

name: api-test-report-\${{ matrix.environment }}

```

path: reports/

# — Job riêng: Contract Testing —
contract-test:
  runs-on: ubuntu-latest
  needs: api-test
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
  with: { node-version: '20' }
    - run: npm ci
    - name: Run Pact Consumer Tests
  run: npm run test:consumer
    - name: Publish Pacts to Broker
  run: npx pact-broker publish ./pacts --consumer-app-version=${{ github.sha }}
  env:
    PACT_BROKER_TOKEN: ${ secrets.PACT_BROKER_TOKEN }
    - name: Can-I-Deploy?
  run: npx pact-broker can-i-deploy --pacticipant web-frontend --version=${{ github.sha }}
  --to-environment production
  env:
    PACT_BROKER_TOKEN: ${ secrets.PACT_BROKER_TOKEN }

```

Kết quả: Mỗi khi có commit/PR, API tests tự động chạy trên cả 2 môi trường song song, sau đó contract test verify tương thích trước khi deploy.

D3. Test Report

HTML Report (newman-reporter-htmlextra):

```
reports/report.html
```

- └─ Summary: Total/Pass/Fail/Skipped
- └─ Per Request: Status, Response Time, Test Results
- └─ Failure Details: Expected vs Actual

JUnit XML (cho CI integration):

```
<testsuite name="API Tests" tests="15" failures="2" time="3.245">
  <testcase name="POST /auth/login - Status 200" time="0.125"/>
  <testcase name="GET /users - Status 200" time="0.089"/>
  <testcase name="GET /users/invalid - Status 404" time="0.067">
    <failure>Expected 404 but got 200</failure>
  </testcase>
</testsuite>
```

7. Phần E — AI Agent trong API & Contract Testing

Đây là phần mở rộng thực tế nhất — phản ánh xu hướng hiện tại của ngành khi AI ngày càng được tích hợp sâu vào quy trình kiểm thử.

E1. Landscape: Các loại AI Agent trong API Testing

AI không chỉ đơn giản là "gợi ý test case" — trong thực tế, AI Agent đang được áp dụng ở nhiều lớp khác nhau trong quy trình testing:

AI trong API Testing

- |
- └─ Level 1 – AI Copilot (hỗ trợ human)
 - | └─ Generate test cases từ spec (ChatGPT, Claude, Copilot)
 - | └─ Viết Postman/Jest script boilerplate
 - | └─ Review OpenAPI spec, tìm lỗi thiết kế
- |
- └─ Level 2 – AI-Augmented Tools (tool tích hợp AI)

- | └─ Postman AI (Postbot): tự động sinh test script từ response
- | └─ Swagger Inspector AI: phân tích endpoint tự động
- | └─ REST Assured + AI code generation
- | └─ Schemathesis: tự động fuzz-test từ OpenAPI spec
- |
- | └─ Level 3 – AI Agent Autonomous (tự động hoàn toàn)
- | └─ Tự khám phá API endpoint từ source code
- | └─ Tự sinh contract từ interaction logs
- | └─ Tự detect regression khi response thay đổi
- | └─ Tự recommend fix khi test fail
- |
- | └─ Level 4 – AI trong CI/CD Pipeline
- | └─ AI phân tích test report → tìm pattern lỗi
- | └─ AI quyết định rollback khi phát hiện anomaly
- | └─ AI tự update test suite khi API version thay đổi

E2. AI Generate Test Cases — Thực tế & Hiệu quả

E2.1 Prompt mẫu — Generate từ OpenAPI Spec

Dưới đây là OpenAPI spec cho endpoint POST /api/v1/users:

requestBody:

required: true

content:

application/json:

schema:

type: object

required: [name, email, age]

```
properties:
  name:
    type: string
    minLength: 1
    maxLength: 50
  email:
    type: string
    format: email
  age:
    type: integer
    minimum: 1
    maximum: 120
  role:
    type: string
    enum: [user, admin]
    default: user
---
```

Hãy thiết kế test cases đầy đủ theo các kỹ thuật:

1. Positive testing (happy path)
2. Negative testing (thiếu field bắt buộc, sai type)
3. Boundary Value Analysis cho name (length) và age
4. Equivalence Partitioning cho email
5. Security testing (mass assignment – thêm field không có trong spec)

Format: Bảng markdown với TC_ID | Input (JSON) | Expected Status | Technique | Ghi chú

E2.2 Postbot — AI tích hợp trong Postman

Postman đã tích hợp **Postbot** (AI assistant) vào phiên bản mới:

Cách dùng Postbot:

1. Gửi một request → nhận response
2. Click "Postbot" (icon AI) trong tab Tests
3. Chọn: "Add tests for this request"
4. Postbot tự động sinh pm.test() dựa trên response thực tế
5. Review và adjust các test được sinh ra

Ví dụ: Postbot sinh test script từ response:

```
// Response thực tế: { "id": 42, "name": "Alice", "email": "a@test.com", "role": "user" }  
  
// Postbot tự động sinh:  
  
pm.test('Status code is 200', () => pm.response.to.have.status(200));  
  
pm.test('Response time < 1000ms', () =>  
  pm.expect(pm.response.responseTime).to.be.below(1000));  
  
pm.test('id is a number', () => pm.expect(pm.response.json().id).to.be.a('number'));  
  
pm.test('name is a string', () => pm.expect(pm.response.json().name).to.be.a('string'));  
  
pm.test('email contains @', () => pm.expect(pm.response.json().email).to.include('@'));  
  
pm.test('role is valid', () => pm.expect(['user',  
  'admin']).to.include(pm.response.json().role));
```

E2.3 Schemathesis — AI-Powered API Fuzzing

Schemathesis là tool tự động generate hàng trăm test case từ OpenAPI spec, áp dụng kỹ thuật property-based testing:

```
# Cài đặt  
  
pip install schemathesis  
  
# Tự động fuzz API từ OpenAPI spec  
  
schemathesis run http://localhost:3000/api/openapi.json \  
  --checks all \  
  --auth-type=bearer \  
  --auth="your-token-here" \  

```



```
--report schemathesis-report.html
```

```
# Schemathesis tự sinh:
```

```
# - Hàng trăm combinations input (BVA, EP, null, empty...)
```

```
# - Kiểm tra 5xx responses (server error)
```

```
# - Kiểm tra response schema consistency
```

```
# - Detect IDOR bằng cách thay đổi path parameters
```

E3. AI trong Contract Testing

E3.1 AI tự động sinh Pact Contract từ API logs

Thay vì viết contract thủ công, AI có thể phân tích HTTP traffic logs và sinh Pact file:

Prompt mẫu — AI sinh Pact contract từ HTTP log:

Dưới đây là HTTP interaction log giữa web-frontend và user-service:

Request:

GET /users/42

Authorization: Bearer eyJhbGci...

Response:

200 OK

Content-Type: application/json

Body: {"id": 42, "name": "Alice", "email": "alice@test.com", "role": "user"}

Hãy:

1. Viết Pact interaction cho interaction này (Pact V3 format, JavaScript)
2. Dùng MatchersV3 phù hợp (integer, string, regex) thay vì exact matching
3. Đặt tên Provider State hợp lý

4. Giải thích tại sao chọn matcher đó thay vì exact match

E3.2 AI Review Contract Compatibility

Tôi có 2 OpenAPI spec:

[Consumer Expected Spec – frontend mong đợi]

Paste OpenAPI YAML ở đây...

[Provider Actual Spec – backend hiện tại cung cấp]

Paste OpenAPI YAML ở đây...

Hãy:

1. Phát hiện các breaking changes (field bị xóa, type thay đổi)
2. Phát hiện các non-breaking changes (field thêm mới, optional field)
3. Đánh giá mức độ tương thích: COMPATIBLE / INCOMPATIBLE / PARTIAL
4. Gợi ý cách provider fix để không break consumer

E4. AI-Powered API Exploration & Fuzzing

Khác với testing từ spec, AI có thể **khám phá API** mà không cần biết spec trước:

Workflow AI API Exploration:

1. Cung cấp base URL
2. AI tự gửi OPTIONS request để khám phá methods
3. AI phân tích response patterns
4. AI tự sinh test cases cho hidden endpoints
5. AI báo cáo: endpoints nào trả về data nhạy cảm không cần auth?

Tool thực tế:

- **RESTler** (Microsoft): Stateful REST API fuzzer, tự phát hiện sequence attacks

- **CATS** (Continuous API Testing): Fuzzer tích hợp OpenAPI, 100+ fuzz strategies
- **Dredd**: API description validation tool

```
# Ví dụ dùng CATS

cats --contract openapi.yaml --server http://localhost:3000 \

--fuzzers=RandomFuzzer,BoundaryValueFuzzer,NullFuzzer \

--report report.html
```

E5. Tích hợp AI vào CI/CD Pipeline

```
# .github/workflows/ai-assisted-api-tests.yml

jobs:
  ai-test-generation:
    runs-on: ubuntu-latest

    steps:
      - name: Generate test cases with AI
        run: |
          # Dùng OpenAI API để sinh test data từ schema
          python scripts/generate_test_data.py \
            --schema openapi.yaml \
            --output testdata/generated.csv \
            --count 50

          env:
            OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY }

      - name: Run Newman với AI-generated test data
        run: |
          newman run collection.json \
            -d testdata/generated.csv \
            -r cli,junit --reporter-junit-export reports/junit.xml
```

```
- name: AI analyze failures
if: failure()
run: |
# Gửi junit.xml cho AI phân tích pattern lỗi
python scripts/ai_analyze_failures.py \
--report reports/junit.xml \
--output reports/ai_analysis.md
env:
OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY }
```

E6. Prompt Engineering cho API Testing

Template 1 — Generate Test Cases từ Spec

Role: Bạn là senior QA engineer với 10 năm kinh nghiệm API testing.

Task: Thiết kế test cases cho endpoint sau.

Context: [mô tả hệ thống, business rules]

Constraints:

- Dùng kỹ thuật: BVA, EP, Negative testing, Security testing
- Format: Bảng markdown TC_ID | Input | Expected | Technique | Priority
- Priority: Critical / High / Medium / Low

Endpoint spec: [paste spec ở đây]

Template 2 — Generate Postman Script

Role: Senior API automation engineer.

Task: Viết Postman test script (pm API, JavaScript ES6+) cho response:

[paste response JSON]

Constraints:

- Validate status code

- Validate schema (dùng `pm.response.to.have.jsonSchema`)
- Validate business rules: [mô tả rules]
- Lưu token vào environment variable nếu response có `access_token`
- Response time < [X]ms
- Không dùng hardcoded values, dùng `pm.environment.get()`

Template 3 — AI Contract Review

Role: API contract specialist.

Task: So sánh Provider API implementation với Consumer expectations.

Consumer spec (expected): [paste]

Provider spec (actual): [paste]

Output format:

1. Breaking changes: [list]
2. Non-breaking changes: [list]
3. Compatibility verdict: COMPATIBLE / INCOMPATIBLE
4. Recommended fixes for Provider: [list]
5. Recommended workarounds for Consumer: [list]

E7. Giới hạn & Rủi ro khi dùng AI trong Testing

Khía cạnh	AI làm tốt	AI cần human review
Test case generation	Happy path, BVA, EP, Negative từ spec	Business logic phức tạp, domain-specific rules
Script writing	Boilerplate <code>pm.test()</code> , schema validation	Complex chaining, async flows

Khía cạnh	AI làm tốt	AI cần human review
Security testing	Liệt kê OWASP categories, common payloads	Zero-day exploits, context-specific attacks
Contract testing	Sinh basic interactions từ logs	Stateful sequences, race conditions
Analysis	Pattern recognition trong large test suites	Root cause analysis đòi hỏi domain knowledge
Accuracy	Kết quả tốt với spec rõ ràng	Có thể hallucinate API không tồn tại
Security risk	—	KHÔNG paste prod credentials/API keys vào AI

Cảnh báo thực tế: AI-generated test cases KHÔNG thay thế được tư duy của tester. AI không biết business context, không biết user journey, không biết legacy behavior. AI là **accelerator**, không phải **replacement**.

8. Phần F — Thực hành (Hands-on Lab)

F1. Sample API

Gợi ý API mẫu (mã nguồn mở):

Lựa chọn	Ưu điểm	Link
JSONPlaceholder	Sẵn có online, không cần setup	https://jsonplaceholder.typicode.com

Lựa chọn	Ưu điểm	Link
Reqres.in	Có auth, nhiều endpoint	https://reqres.in
Express + SQLite	Tự build, full control, có thể thêm auth	Tự build
Node.js CRUD API	Template sẵn có trên GitHub	Tìm trên github

Khuyến nghị: Tự build một API nhỏ bằng Express + JWT để:

- Kiểm soát hoàn toàn các endpoint
- Có thể demo authentication
- Demo contract testing thực tế

F2. Bài tập thực hành cho học viên

Bài 1: Postman Basic (15 phút)

1. Import collection từ repo GitHub
2. Thiết lập Environment với baseUrl
3. Chạy các request có sẵn, xem response
4. Thêm test script cho 1 endpoint tự chọn

Bài 2: Data-Driven Testing (15 phút)

1. Tạo file testdata.csv với 5 bộ dữ liệu
2. Chạy Collection Runner với file CSV
3. Xem kết quả – request nào pass/fail

Bài 3: Contract Testing (nếu có thời gian)

1. Đọc Pact contract file có sẵn trong repo

2. Hiểu interaction: consumer expects gì
3. Chạy Pact verification test

9. Phân công nội dung cho thành viên

Đề xuất phân công

MSSV	Thành viên	Phần phụ trách	Sản phẩm cần nộp
23127115	Mạch Quốc Tấn	Phần A2 (Kỹ thuật test case) + A3 (Test theo loại endpoint) + Tổng hợp scope	Tài liệu kỹ thuật test case, slide A
23127065	Ngô Nguyễn Thế Khoa	Phần B (Contract Testing: CDCT, OpenAPI, Pact, Bi-directional)	Tài liệu Contract Testing, slide B
23127211	Nguyễn Lê Hồ Anh Khoa	Phần C (Postman: Collection, Env, Variables, Test Script, Pre-request, Mock)	Video demo Postman (4-5 kịch bản)
23127148	Ân Tiến Nguyễn An	Phần C5 (Data-Driven) + Phần E (AI-Assisted Testing) + Tool survey	Video demo DDT + tài liệu AI prompts
23127152	Nguyễn Tuấn Anh	Phần D (Newman + GitHub Actions CI/CD) + F (Setup lab repo) + Outline slide/báo cáo	Video demo CI/CD pipeline + repo GitHub

10. Danh sách Video Demo cần chuẩn bị

Mỗi video là một kịch bản độc lập, không gộp chung.

#	Video	Nội dung	Thời lượng dự kiến	Phụ trách
V1	Postman Basics	Import collection, setup Env, chạy GET/POST/PUT/DELETE	3-5 phút	Anh Khoa
V2	Test Scripts	Viết pm.test assertions, validate schema, lưu variable	4-6 phút	Anh Khoa
V3	Authentication Testing	Test no-token, expired, valid, IDOR, RBAC	5-7 phút	Anh Khoa / Tấn
V4	Data-Driven Testing	Tạo CSV, chạy Collection Runner với nhiều bộ data	4-5 phút	Nguyễn An
V5	Newman CLI	Export collection, chạy Newman, xem HTML report	4-5 phút	Tuấn Anh
V6	GitHub Actions CI/CD	Push code → Actions chạy → kết quả pass/fail	5-7 phút	Tuấn Anh
V7	Contract Testing (Pact)	Giải thích workflow, demo verify contract	5-8 phút	Thế Khoa
V8	AI-Assisted Testing	Demo dùng AI generate test case, review spec	3-4 phút	Nguyễn An

11. Tiêu chí chấm điểm & Checklist hoàn thành

Checklist Nội dung

- ☐ Trình bày được **định nghĩa** và **vai trò** của API Testing
- ☐ Phân biệt API **có/không có authentication**
- ☐ Demo **ít nhất 3 kỹ thuật test case** (Positive, Negative, BVA, DDT)
- ☐ Demo **đầy đủ tính năng Postman**: Collection, Env, Variable, Script, Runner
- ☐ Demo **Postbot AI** sinh test script tự động
- ☐ Giải thích **Contract Testing** và **tại sao cần** nó trong microservices
- ☐ Trình bày **Pact code** Consumer + Provider verify (không chỉ JSON file)
- ☐ Demo **automation** với Newman + CI/CD (GitHub Actions, environment matrix)
- ☐ Trình bày **AI Agent landscape** trong API Testing (3 levels trở lên)
- ☐ Demo **ít nhất 1 AI tool thực tế** (Postbot, Schemathesis, hoặc ChatGPT prompt)
- ☐ Giải thích **giới hạn & rủi ro** khi dùng AI trong testing
- ☐ Có **phần thực hành** để học viên tự làm trong lớp

Checklist Sản phẩm

- ☐ Slide đầy đủ, có ví dụ cụ thể
- ☐ Tối thiểu 5 video demo (mỗi kịch bản riêng biệt)
- ☐ Video V8 (AI-Assisted) thể hiện rõ workflow AI → test → result
- ☐ Báo cáo viết chi tiết (giải thích lý do, trade-offs)
- ☐ Repo GitHub có cấu trúc rõ ràng, có README hướng dẫn
- ☐ File testdata (CSV/JSON) cho bài tập
- ☐ AI Audit Report đầy đủ

Rubric tự đánh giá

Tiêu chí	Điểm tối đa	Ghi chú
Lý thuyết đầy đủ, chính xác	15	API + Contract testing theory
Kỹ thuật test case đa dạng	20	BVA, DDT, Security (với script demo), Auth
Demo Postman phong phú	20	Tối thiểu 4 chức năng, có Postbot AI
Contract Testing thực chiến	15	Pact code (consumer + provider), OpenAPI valid

Tiêu chí	Điểm tối đa	Ghi chú
Automation & CI/CD	15	Newman + GitHub Actions (env matrix)
AI Agent trong Testing	10	Landscape, tools, demo thực tế, giới hạn
Thực hành học viên	5	Lab có thể làm được trong lớp

Tài liệu tham khảo

API Testing

1. [API Testing Theory.md](#) — Lý thuyết nền đã nghiên cứu
2. [OWASP API Security Top 10 \(2023\)](#)
3. [Newman CLI Docs](#) — CLI runner cho Postman
4. [Postman Learning Center](#) — Tài liệu chính thức
5. [OpenAPI Specification 3.0](#) — API Design standard

Contract Testing

1. [Pact Documentation](#) — Consumer-Driven Contract Testing
2. [Pact JS GitHub](#) — JavaScript implementation
3. [PactFlow — Bi-directional Contract Testing](#)
4. [Pact Workshop JS](#) — Tutorial thực hành

AI trong Testing

1. [Postman Postbot Documentation](#) — AI assistant trong Postman
2. [Schemathesis](#) — Property-based API testing từ OpenAPI spec
3. [CATS — Continuous API Testing for Swagger](#) — OpenAPI fuzzer
4. [RESTler \(Microsoft\)](#) — Stateful REST API fuzzer
5. [GitHub Copilot for Testing](#) — AI code completion

CI/CD & Automation

1. [GitHub Actions Documentation](#) — CI/CD setup
2. [Newman Reporter HTMLExtra](#) — HTML report

Tài liệu này được tạo và cải thiện bởi AI (Claude Sonnet 4.6 Thinking) dựa trên nội dung trao đổi tại Script.md và nghiên cứu chuyên sâu. Phiên bản 2.0 bổ sung: AI Agent landscape, Pact code hoàn chỉnh, Security test scripts, Environment matrix CI/CD. Đã được xem xét và điều chỉnh bởi Mạch Quốc Tấn (23127115).